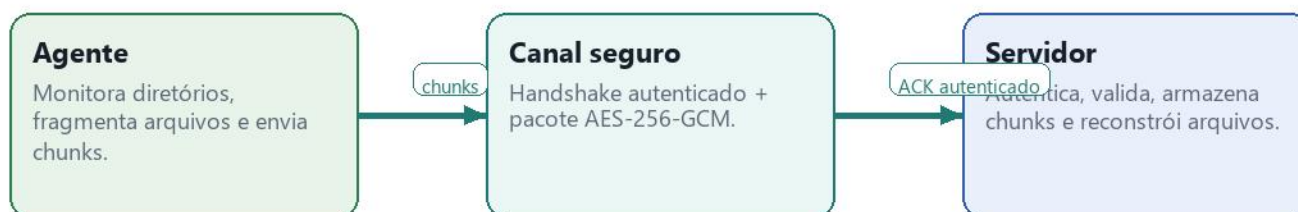


TransferUDT

Arquitetura, funcionamento e caso de uso

Visão técnica da aplicação de transferência segura por UDT, incluindo componentes, fluxo criptográfico, controles de segurança e uma execução end-to-end.



O que este documento cobre

- Arquitetura lógica: Agent, TransferCore, Server, SQLite e armazenamento em disco.
- Fluxo de autenticação, criptografia, validação de integridade e reconstrução.
- Controles adicionados para isolamento por identidade, quotas, filas limitadas e rejeição de links.
- Caso de uso passo a passo com resultado esperado e validação end-to-end.

Arquitetura lógica

AgentUDTC++_v7.2

FileWatcher

Observa data.dirs e rejeita links/reparse points.

FileProcessor

Calcula hash, divide chunks e controla estabilidade.

NetworkManager

Executa handshake, envelope seguro e ACK autenticado.

SQLite local

Estado de filas, chunks e entregas confirmadas.

TransferCore

SecurityHandshake

Challenge-response com client_id e HMAC-SHA256.

SecurePacket

AES-256-GCM, nonce, tag e metadados autenticados.

ThreadPool

Fila com limite para reduzir DoS por conexões pendentes.

Logger/Config

Telemetria e parâmetros de segurança compartilhados.

ServerUDTC++_v3

ServerUDT

Aceita conexões UDT e aplica bind/allowlist de origem.

ClientHandler

Mantém identidade autenticada durante a sessão.

FileReceiver

Valida chunk, quota, hashes e monta arquivo final.

Database + storage

Namespace por client_id e persistência de progresso.

protocolo

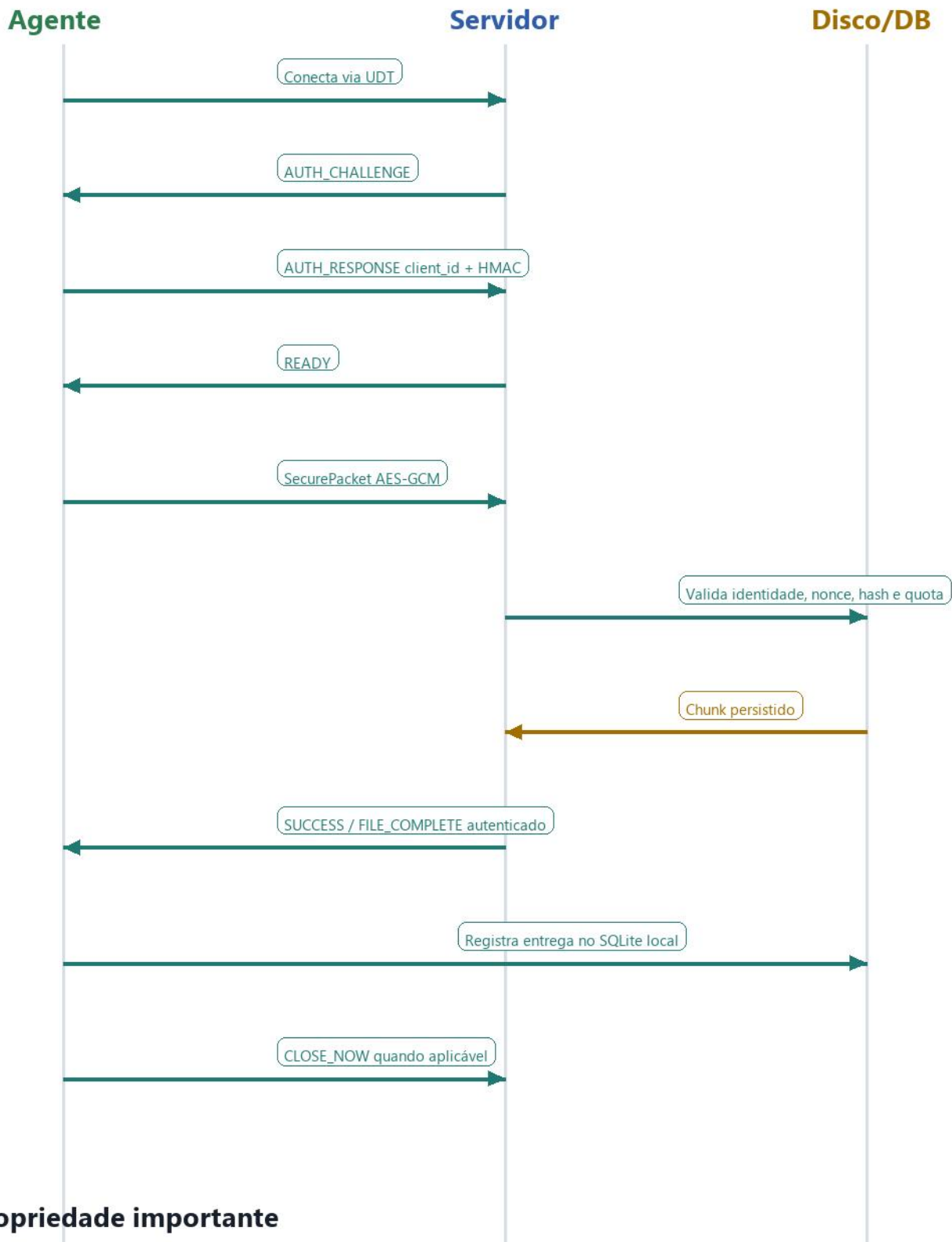
sessão

chunks

Modelo de responsabilidade

- O Agent decide o que enviar e nunca deve seguir caminhos fora dos diretórios monitorados.
- O TransferCore centraliza regras de segurança reutilizadas pelos dois binários.
- O Server é a autoridade de recepção: autentica, aplica quota, grava estado e reconstrói o arquivo.

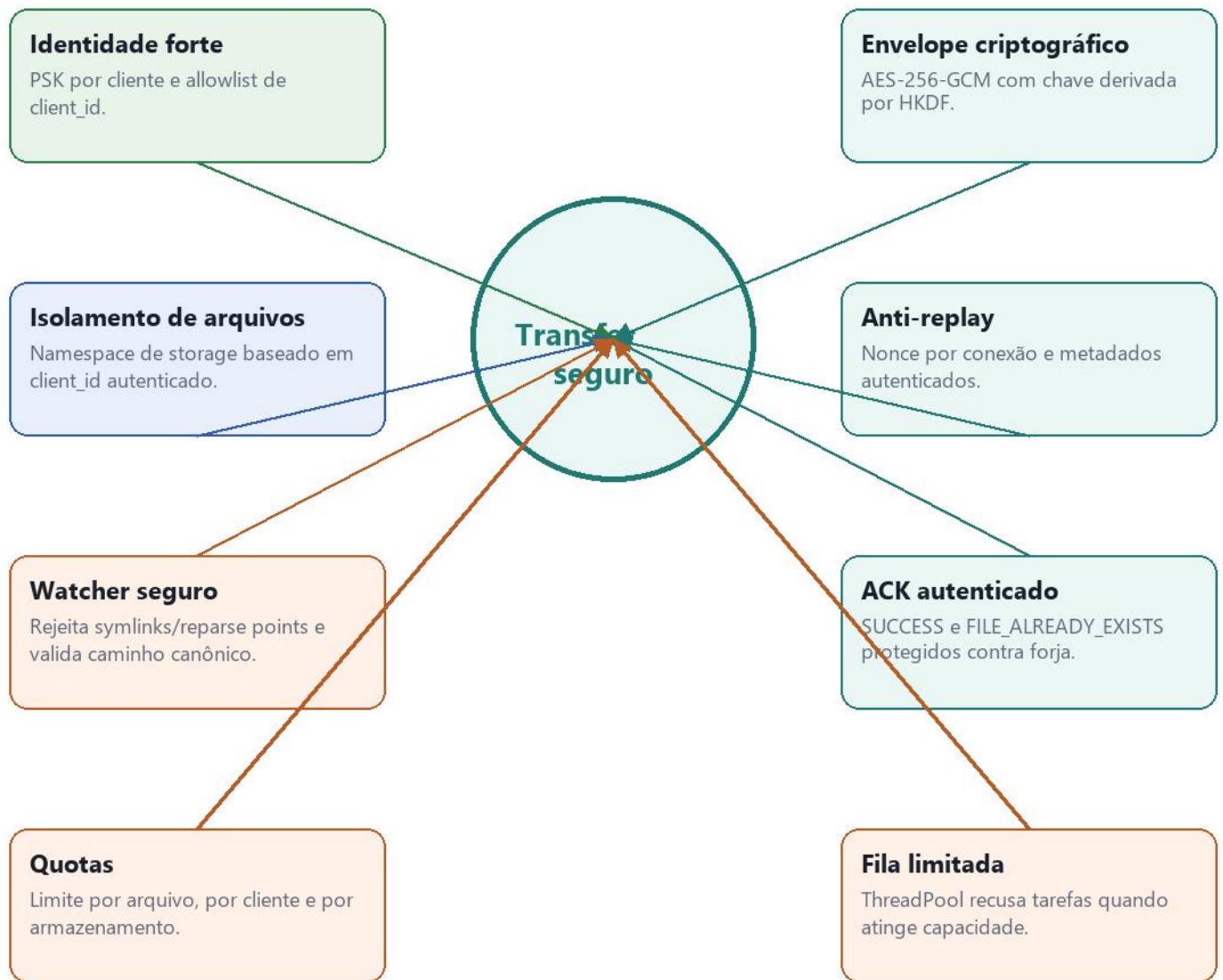
Fluxo seguro de transferência



Propriedade importante

A entrega só é marcada como bem-sucedida depois que a resposta de controle também passa pelo envelope autenticado. Assim, um endpoint falso ou um atacante de rede não consegue forjar SUCCESS sem conhecer a chave da sessão.

Controles de segurança e resiliência



Risco residual conhecido

- A autenticação usa PSK/HMAC, não cadeia de certificados ou mTLS.
- A proteção anti-replay é por conexão; a idempotência e o banco reduzem duplicidade operacional.
- Parâmetros de quota, allowlist e chaves por cliente precisam ser tratados como configuração de produção.

Caso de uso passo a passo

Cenário: um agente em uma estação de produção envia um arquivo gerado localmente para um servidor central, preservando integridade, identidade e isolamento por cliente.

1

Configurar agente

Definir `data.dirs`, `client_id`, PSK do cliente, endereço do servidor e limites de chunk.

2

Detectar arquivo

`FileWatcher` percebe o novo arquivo e aguarda estabilidade antes de enfileirar.

3

Validar caminho

O agente rejeita symlink/reparse point e confirma que o caminho canônico permanece dentro da pasta monitorada.

4

Fragmentar

`FileProcessor` calcula hash, divide em chunks e grava estado local no SQLite.

5

Autenticar

`NetworkManager` abre conexão UDT, responde ao challenge com HMAC e recebe READY.

6

Enviar chunk

O chunk viaja no `SecurePacket`, com metadados e payload autenticados por AES-GCM.

7

Persistir no servidor

`FileReceiver` valida identidade, hash, nonce e quota antes de escrever no namespace do cliente.

8

Confirmar entrega

Servidor envia ACK autenticado; agente registra sucesso e remove pendências quando completo.

Organização dos dados

Namespace por cliente

Arquivos e registros são separados pelo `client_id` autenticado. Isso evita colisão entre clientes atrás do mesmo NAT ou processos diferentes no mesmo host.

server-reconstructed/

— id_e2e-agent/

— agent-watch/

— e2e_payload.bin

— id_outro-cliente/

— agent-watch/

— relatorio.csv



Efeito prático

- Dois agentes com o mesmo IP não competem pelo mesmo arquivo lógico.
- Falhas e reenvios podem retomar pelo estado persistido, sem misturar chunks de outro cliente.
- Rotinas de backup, auditoria e limpeza conseguem operar por cliente.

Validação end-to-end

A validação executada no workspace testou o caminho real de envio e um cenário negativo de quota. Os resultados abaixo resumem o comportamento observado depois das melhorias.

Envio positivo

Arquivo de 24.576 bytes transferido e reconstruído em `server-reconstructed/id_e2e-agent/agent-watch/e2e_payload.bin`. SHA-256 confirmado: 95B6FD038BDCBB4B659313E997E5950E2917C74CEFC2579FE38F7FDE013A3239.

Rejeição por limite

Com `server.max_file_size_mb=1`, um arquivo de 2.097.152 bytes foi rejeitado pelo servidor e nenhum arquivo reconstruído foi produzido.

Testes de unidade e build

Server: 25/25 testes. Agent: 27/27 testes. Builds Debug de Agent e Server executados com sucesso.

Leitura operacional

- O fluxo principal entrega e reconstrói o arquivo esperado.
- A quota atua antes de materializar arquivos grandes indevidos.
- Os testes cobrem os principais controles P1/P2 adicionados nesta rodada.